

Modül 7

Services & DI

Dependency Injection Derinlemesine

Modül: 7 / 30

Ders Sayısı: 8 ders

Seviye: Orta-İleri

Versiyon: Angular v21

Hazırlayan: Claude • Türkçe Angular v21 Eğitimi

Modül 7'ye Hoş Geldin

Bu modül Angular'ın temel taşı olan Dependency Injection (DI) sistemini detaylı inceliyor. Service'ler, provider'lar, injection scope'ları, injection token'ları, multi providers — hepsi.

Bu modülün sonunda yapabileceklerin:

- ✓ DI prensiplerini ve Angular'da nasıl çalıştığını anlamak
- ✓ Service oluşturup farklı scope'larda kullanmak
- ✓ inject() fonksiyonu ile modern DI
- ✓ Provider hierarchy'i yönetmek
- ✓ Injection Token'ları kullanmak
- ✓ Multi providers ile esnek mimari
- ✓ Injection context'i anlamak
- ✓ Factory provider'lar ile dinamik DI

Dependency Injection Nedir?

DI yazılım dünyasının en güçlü pattern'lerinden biri. Önce kavramı, sonra Angular'daki implementasyonu.

❑ Problem — DI Olmadan

```
// ❑ DI olmadan – Tight coupling
class UserComponent {
  private userService: UserService;

  constructor() {
    // Component KENDİ instance'ını yaratıyor
    this.userService = new UserService(
      new HttpClient(
        new HttpHandler()
      )
    );
  }
}

// Sorunlar:
// 1. UserComponent UserService'i KENDİ yarattığı için test ZOR
// 2. UserService'in tüm dependency'lerini bilmek zorunda
// 3. Mock service ile değiştiremezsin
// 4. Singleton'a zorlamak zor
```

❑ Çözüm — DI ile

```
// ✓ DI ile – Loose coupling
class UserComponent {
  constructor(private userService: UserService) {
    // Component sadece "bir UserService istiyorum" diyor
    // Framework instance'ı veriyor
  }
}

// Modern syntax (inject):
class UserComponent {
  private userService = inject(UserService);
}

// Avantajlar:
// 1. Test'te mock service inject edebilirsiniz
// 2. Component, service'in nasıl yaratıldığını bilmiyor
// 3. Otomatik singleton yönetimi
// 4. Loose coupling
```

□ DI'ın 3 Temel Soru

- Ne sağlanacak? → Token (class veya InjectionToken)
- Kim sağlayacak? → Provider (factory, value, class, vs.)
- Nerede sağlanacak? → Injector (scope - root, component)

□ Injector Tree

Angular'da injector'lar ağaç (tree) oluşturur. Bir component'in injector'ı bulamadığı bir token için parent'a sorar.

```
Root Injector (providedIn: 'root')
  ↓
Route Injector (lazy loaded routes)
  ↓
Component Injector (providers: [...])
  ↓
Element Injector (directive providers)

// Inject lookup yukarı doğru gider:
// Component → Route → Root
// İlk bulunan instance kullanılır
```

Service Oluşturma — providedIn

Service'lerin nasıl provide edildiğini ve scope'larını öğrenelim.

□ providedIn: 'root'

En yaygın yöntem. Service tüm uygulama için singleton. Tree-shakable.

```
@Injectable({ providedIn: 'root' })
export class UserService {
  // Tek instance, tüm app'te paylaşılır
}

// CLI ile:
// ng g s services/user
```

□ providers: [] (Component)

Service'i sadece o component ve child'larında kullanmak istersen:

```
@Component({
  selector: 'app-modal',
  providers: [ModalStateService], // ← Sadece bu component için
  ...
})
export class ModalComponent {
  private state = inject(ModalStateService);
}

// Her ModalComponent instance'ı KENDİ ModalStateService'ine sahip
// Bu pattern "local state service" için ideal
```

□ Route-level Providers

Lazy-loaded route'lara özel service'ler:

```
export const routes: Routes = [
  {
    path: 'admin',
    loadComponent: () => import('./admin/admin.component'),
    providers: [
      AdminService, // Sadece /admin route'da
      { provide: API_URL, useValue: '/admin-api' }
    ]
  }
];
```

□ Scope Karşılaştırması

Scope	Singleton?	Use Case
providedIn: root	✓ Tek instance	Genel servisler (Auth, HTTP, vs.)
providedIn: 'platform'	✓ Tek (ms)	Mikrofrontend uygulamalar arası
Component providers	✗ Component bazlı	Component-local state
Route providers	✗ Route bazlı	Feature-specific services

inject() Function — Modern DI

Constructor injection vs inject() — modern Angular tercihi.

❑ Eski Yöntem: Constructor Injection

```
@Component({...})
export class UserComponent {
  constructor(
    private userService: UserService,
    private route: ActivatedRoute,
    private router: Router,
    private http: HttpClient,
  ) {}
}
```

❑ Modern: inject()

```
@Component({...})
export class UserComponent {
  private userService = inject(UserService);
  private route = inject(ActivatedRoute);
  private router = inject(Router);
  private http = inject(HttpClient);

  constructor() {
    // Artık temiz constructor
  }
}
```

⚡ inject() Avantajları

- ✓ Inheritance kolay: Parent class constructor parametreleri sorun değil
- ✓ Mixin/Composable functions: Constructor dışında DI yapabilirsin
- ✓ Optional inject: { optional: true }
- ✓ Daha az boilerplate: private modifier syntax kalkıyor
- ✓ Type inference: inject(MyService) → MyService

❑ inject() Options

```
// Optional - service yoksa null
private logger = inject(LoggerService, { optional: true });

// Self - sadece kendi injector'unda ara
private state = inject(LocalStateService, { self: true });

// SkipSelf - kendi injector'unu atla, parent'a git
private parentState = inject(StateService, { skipSelf: true });

// Host - host component'e kadar ara (directive'de)
private parent = inject(ParentComponent, { host: true });
```

□ Composable Functions (Custom Hooks)

inject() sayesinde React-style composable function'lar yazabilirsiniz:

```
// Custom composable
export function useUser() {
  const userService = inject(UserService);
  const route = inject(ActivatedRoute);

  const userId = toSignal(
    route.paramMap.pipe(map(p => p.get('id'))),
    { initialValue: null }
  );

  const user = httpResource<User>(() =>
    userId() ? `/api/users/${userId()}` : undefined
  );

  return { userId, user };
}

// Component'te kullanım:
@Component({...})
export class UserDetailsComponent {
  protected user = useUser(); // Composable
}
```


Provider Türleri

useClass, useValue, useFactory, useExisting — DI'n 4 silahşörü.

1 useClass — Class Provider

```
// Implicit (kısa yol)
providers: [UserService]
// = { provide: UserService, useClass: UserService }

// Explicit - farklı implementation
providers: [
  { provide: UserService, useClass: MockUserService } // Test için
]
```

2 useValue — Value Provider

```
// Config objesi inject et
providers: [
  {
    provide: APP_CONFIG,
    useValue: {
      apiUrl: 'https://api.example.com',
      timeout: 5000,
      retries: 3
    }
  }
]

// Component'te:
const config = inject(APP_CONFIG);
console.log(config.apiUrl);
```

3 useFactory — Factory Provider

```
// Dynamic provider - inject'ler kullanarak yarat
providers: [
  {
    provide: ApiClient,
    useFactory: (config: AppConfig, http: HttpClient) => {
      return new ApiClient(config.apiUrl, http);
    },
    deps: [APP_CONFIG, HttpClient]
  }
]

// Modern syntax:
providers: [
  {
    provide: ApiClient,
    useFactory: () => {
      const config = inject(APP_CONFIG);
      const http = inject(HttpClient);
      return new ApiClient(config.apiUrl, http);
    }
  }
]
```

4. useExisting — Alias Provider

Aynı instance'ı iki farklı token üzerinden erişilebilir kılar:

```
abstract class AbstractLogger {
  abstract log(message: string): void;
}

class ConsoleLogger extends AbstractLogger {
  log(msg: string) { console.log(msg); }
}

providers: [
  ConsoleLogger,
  { provide: AbstractLogger, useExisting: ConsoleLogger }
  // AbstractLogger ile inject edilen herkes
  // SAME ConsoleLogger instance'ını alır
]
```

InjectionToken

Class değil bir şey inject etmek için: config object, primitive, function vs.

❑ Neden InjectionToken?

Class'ları inject ederken class ismi token görevi görüyor. Ama config object veya primitive değer inject ederken bir identifier lazım. İşte bu InjectionToken.

❑ InjectionToken Oluşturma

```
import { InjectionToken } from '@angular/core';

// 1. Token tanımla
export const API_URL = new InjectionToken<string>('API_URL');

export const APP_CONFIG = new InjectionToken<AppConfig>('APP_CONFIG', {
  providedIn: 'root',
  factory: () => ({
    apiUrl: 'https://api.default.com',
    timeout: 5000
  })
});

// 2. Provide et
providers: [
  { provide: API_URL, useValue: 'https://api.example.com' }
]

// 3. Inject et
const apiUrl = inject(API_URL);
```

❑ Pratik Use Cases

Use Case 1: Environment-Based Config

```

export const API_BASE = new InjectionToken<string>('API_BASE');

// app.config.ts
providers: [
  {
    provide: API_BASE,
    useValue: environment.production
      ? 'https://api.prod.com'
      : 'http://localhost:3000'
  }
]

```

Use Case 2: Feature Toggle

```

export const FEATURES = new InjectionToken<FeatureFlags>('FEATURES');

interface FeatureFlags {
  newDashboard: boolean;
  betaSearch: boolean;
  experimentalUI: boolean;
}

providers: [
  {
    provide: FEATURES,
    useFactory: () => {
      const env = inject(EnvironmentService);
      return env.loadFeatureFlags();
    }
  }
]

// Component'te
const features = inject(FEATURES);
if (features.newDashboard) { ... }

```

Use Case 3: window/document Inject

```

// SSR uyumlu olmak için DOCUMENT inject et
import { DOCUMENT } from '@angular/common';

@Component({...})
export class MyComponent {
  private doc = inject(DOCUMENT);

  ngOnInit() {
    this.doc.title = 'My App';
    // window yerine: this.doc.defaultView
  }
}

```

Multi Providers

Aynı token için birden fazla provider. Plugin/extension pattern için ideal.

❑ Multi Provider Nedir?

Normal provider: aynı token için sadece bir instance verir.

Multi provider: aynı token için array verir, birden fazla provider birikir.

❑ Örnek: Validators Array

```
export const FORM_VALIDATORS = new InjectionToken<Validator[]>('FORM_VALIDATORS');

// Module 1
providers: [
  { provide: FORM_VALIDATORS, useValue: emailValidator, multi: true }
]

// Module 2 (ayrı yerde)
providers: [
  { provide: FORM_VALIDATORS, useValue: passwordValidator, multi: true }
]

// Module 3
providers: [
  { provide: FORM_VALIDATORS, useValue: phoneValidator, multi: true }
]

// Inject edince array gelir
const validators = inject(FORM_VALIDATORS);
// [emailValidator, passwordValidator, phoneValidator]
```

❑ Pratik: HTTP Interceptor Pattern

Angular'ın HttpInterceptor'ı multi provider üzerine kurulu:

```
// Multiple interceptors
providers: [
  provideHttpClient(
    withInterceptors([
      authInterceptor,
      loggingInterceptor,
      errorInterceptor,
      cacheInterceptor,
    ])
  )
]

// Hepsi sırayla çalışır
```

□ Pratik: Plugin System

```
// Plugin interface
export interface ToolbarPlugin {
  name: string;
  render(): TemplateRef<unknown>;
}

export const TOOLBAR_PLUGINS = new
InjectionToken<ToolbarPlugin[]>('TOOLBAR_PLUGINS');

// app.config.ts - merkezi register
providers: [
  { provide: TOOLBAR_PLUGINS, useClass: SearchPlugin, multi: true },
  { provide: TOOLBAR_PLUGINS, useClass: NotificationPlugin, multi: true },
  { provide: TOOLBAR_PLUGINS, useClass: UserMenuPlugin, multi: true },
]

// Toolbar component'te:
@Component({...})
export class ToolbarComponent {
  plugins = inject(TOOLBAR_PLUGINS);

  // Template'te:
  // @for (plugin of plugins; track plugin.name) {
  //   <ng-container *ngTemplateOutlet="plugin.render()" />
  // }
}
```

Injection Context

`inject()` her yerde çağrılmaz. Injection context'i anlamak gerek.

□ Injection Context Nedir?

Angular `inject()`'in nerede çağrıldığını bilmek zorunda. Çünkü hangi injector kullanılacağı bağlama göre değişir.

□ Inject() Bu Yerlerde Çalışır

- ✓ Constructor: Class constructor içinde
- ✓ Class property initializer: `private x = inject(Y)`
- ✓ @Injectable factory: `{ providedIn: 'root', factory: () => inject(...) }`
- ✓ `runInInjectionContext()`: Explicit context yarat
- ✓ Router resolvers/guards (functional): Bunlar context'tedir
- ✓ HTTP Interceptors (functional): Bunlar da context'tedir

□ Inject() Bu Yerlerde Çalışmaz

- ✗ Regular method body: Method çağrıldığında context yok
- ✗ Async fonksiyonlardan sonra: `await` sonrası context kaybolur
- ✗ `setTimeout` callback: Async, context yok
- ✗ Promise callback: Async, context yok
- ✗ Subscribe callback: Async, context yok

```

@Component({...})
export class MyComponent {
  // ✓ Property initializer - context VAR
  service = inject(UserService);

  constructor() {
    // ✓ Constructor - context VAR
    const router = inject(Router);

    setTimeout(() => {
      // ✗ Async callback - context YOK
      // inject(Router); → Error!
    });
  }

  someMethod() {
    // ✗ Regular method - context YOK
    // inject(Router); → Error!
  }
}

```

✗ runInInjectionContext()

Manuel context yaratmak için:

```

import { runInInjectionContext, inject, Injector } from '@angular/core';

@Component({...})
export class MyComponent {
  private injector = inject(Injector);

  someMethod() {
    runInInjectionContext(this.injector, () => {
      // Burada inject() ÇALIŞIR
      const router = inject(Router);
      router.navigate(['/home']);
    });
  }
}

```


Service Patterns — Production Best Practices

Service'leri profesyonel projelerde nasıl yapılandırıyoruz?

□ Pattern 1: State Service

```
@Injectable({ providedIn: 'root' })
export class TodoService {
  // Private writable
  private _todos = signal<Todo[]>([]);
  private _filter = signal<Filter>('all');

  // Public readonly
  readonly todos = this._todos.asReadonly();
  readonly filter = this._filter.asReadonly();

  // Derived state
  readonly filteredTodos = computed(() => {
    const all = this._todos();
    const f = this._filter();
    return f === 'all' ? all : all.filter(t => t.completed === (f === 'completed'));
  });

  readonly stats = computed(() => ({
    total: this._todos().length,
    active: this._todos().filter(t => !t.completed).length,
    completed: this._todos().filter(t => t.completed).length,
  }));

  // Actions
  add(todo: CreateTodo) { ... }
  remove(id: string) { ... }
  toggle(id: string) { ... }
  setFilter(f: Filter) { this._filter.set(f); }
}
```

□ Pattern 2: API Service

```
@Injectable({ providedIn: 'root' })
export class UserApiService {
  private http = inject(HttpClient);
  private apiUrl = inject(API_BASE);

  getUsers() {
    return this.http.get<User[]>(`${this.apiUrl}/users`);
  }

  getUser(id: string) {
    return this.http.get<User>(`${this.apiUrl}/users/${id}`);
  }

  createUser(user: CreateUserDto) {
    return this.http.post<User>(`${this.apiUrl}/users`, user);
  }

  updateUser(id: string, changes: Partial<User>) {
    return this.http.patch<User>(`${this.apiUrl}/users/${id}`, changes);
  }

  deleteUser(id: string) {
    return this.http.delete(`${this.apiUrl}/users/${id}`);
  }
}
```

□ Pattern 3: Facade Service

Facade: Birden çok service'i tek bir API'ye sarar. Component karmaşıklığı azalır.

```

@Injectables({ providedIn: 'root' })
export class CheckoutFacade {
  private cart = inject(CartService);
  private payment = inject(PaymentService);
  private order = inject(OrderService);
  private notifications = inject(NotificationService);

  // Tek bir method - tüm checkout flow
  async checkout(): Promise<Order> {
    const items = this.cart.items();
    const paymentResult = await this.payment.process(items);

    if (!paymentResult.success) {
      this.notifications.error('Ödeme başarısız');
      throw new Error('Payment failed');
    }

    const order = await this.order.create({ items, payment: paymentResult });
    this.cart.clear();
    this.notifications.success('Sipariş alındı!');

    return order;
  }
}

// Component sadece bu service'i bilir
@Component({...})
export class CheckoutComponent {
  private checkout = inject(CheckoutFacade);
}

```

□ Pattern 4: Mock/Test Provider

```

// Production
providers: [UserApiService]

// Test
providers: [
  { provide: UserApiService, useClass: MockUserApiService }
]

// Mock implementation
@Injectable()
export class MockUserApiService {
  getUsers() {
    return of([
      { id: '1', name: 'Test User' }
    ]);
  }
  // ... diğer mock metodlar
}

```

□ Pattern 5: Functional Service (Modern)

Class yerine fonksiyonlar — modern Angular trendi:

```
// Functional helper
export function createTodoStore() {
  const todos = signal<Todo[]>([]);
  const filter = signal<Filter>('all');

  return {
    todos: todos.asReadonly(),
    filter: filter.asReadonly(),
    filtered: computed(() => /* ... */),

    add: (todo: Todo) => todos.update(t => [...t, todo]),
    remove: (id: string) => todos.update(t => t.filter(x => x.id !== id)),
    setFilter: (f: Filter) => filter.set(f),
  };
}

// Provider olarak
export const TODO_STORE = new InjectionToken('TODO_STORE', {
  providedIn: 'root',
  factory: createTodoStore
});

// Inject
const store = inject(TODO_STORE);
store.add(newTodo);
store.filtered();
```

Modül 7 Özeti

Bu modülde Angular'ın temel taşı DI sistemini detaylı işledik.

Neler Öğrendin?

- ✓ DI prensiplerini ve tree yapısını
- ✓ 4 provider türünü (useClass, useValue, useFactory, useExisting)
- ✓ providedIn scope'larını
- ✓ inject() function ve modern DI
- ✓ InjectionToken kullanımı
- ✓ Multi providers ile plugin pattern
- ✓ Injection context kuralları
- ✓ 5 farklı service pattern (State, API, Facade, Mock, Functional)

Sıradaki Modül

Modül 8 — Routing & Navigation. Route configuration, guards, resolvers, lazy loading, route data, child routes ve daha fazlası.